

操作系统

第二章 进程管理

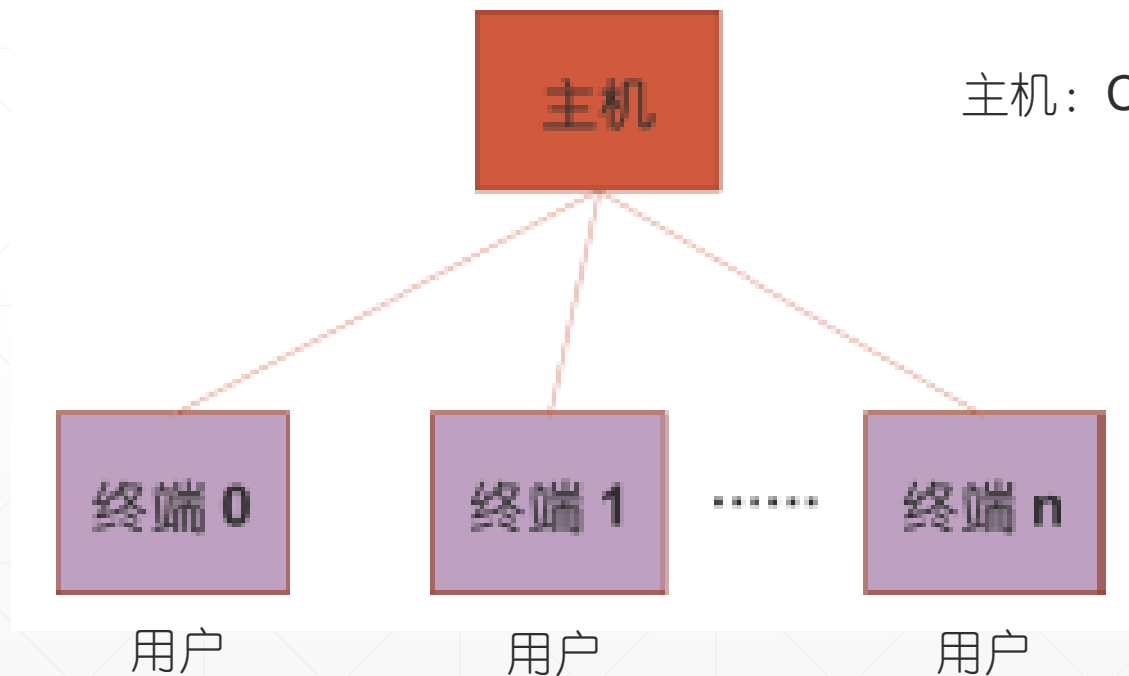
第二部分 (3) 系统运行过程、进程树 和 进程终止

同济大学计算机系



Part 1 Unix系统的运行过程

一、多用户Unix系统，用户与系统的交互——硬件



主机：CPU+内存+磁盘+所有硬件设备和软件资源

终端是人机接口

用户与Unix系统的交互

- 使用一个终端
- 登录 (login) 系统。
- 提交作业 (submit job)，观察作业运行结果，用ctrl+c等等组合键向自己提交的作业发信号，对其运行过程进行控制。
- 离开系统前logout。

- 一次会话，session: uid, tty, shell进程。
- 每执行一个应用程序，系统派发一个进程。
- 应用程序执行完毕，进程终止。

Shell进程

```
i = fork( );  
if(i)  
    .....  
else  
    exec(argv[0], argv);
```

Bochs for Windows - Display

A: B: CD USER Copy Paste snapshot CONFIG Reset SUSPEND Power

```
[/]#cd bin
[/bin]#ls
Directory '/bin':
cat      copyfile      cp      date      echo      fork      forks      ls      malloc
mkdir    newsig    perf     rm      showStack shutdown  sig      sigTest
stack    test      testSTDOUT trace    trivialProg tryExec
```

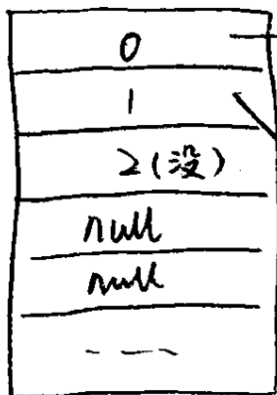
用户与shell进程交互，提交命令行

用户与ls进程交互

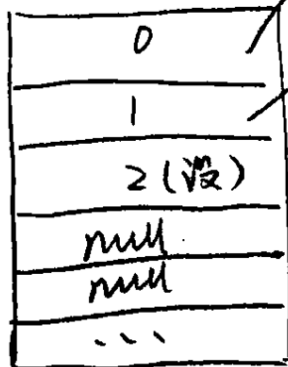
tty

tty使用协议

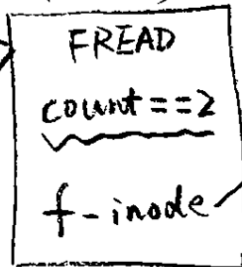
Shell进程



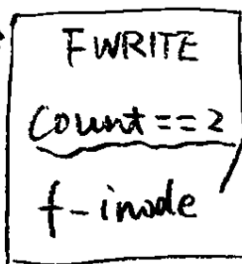
Copy进程



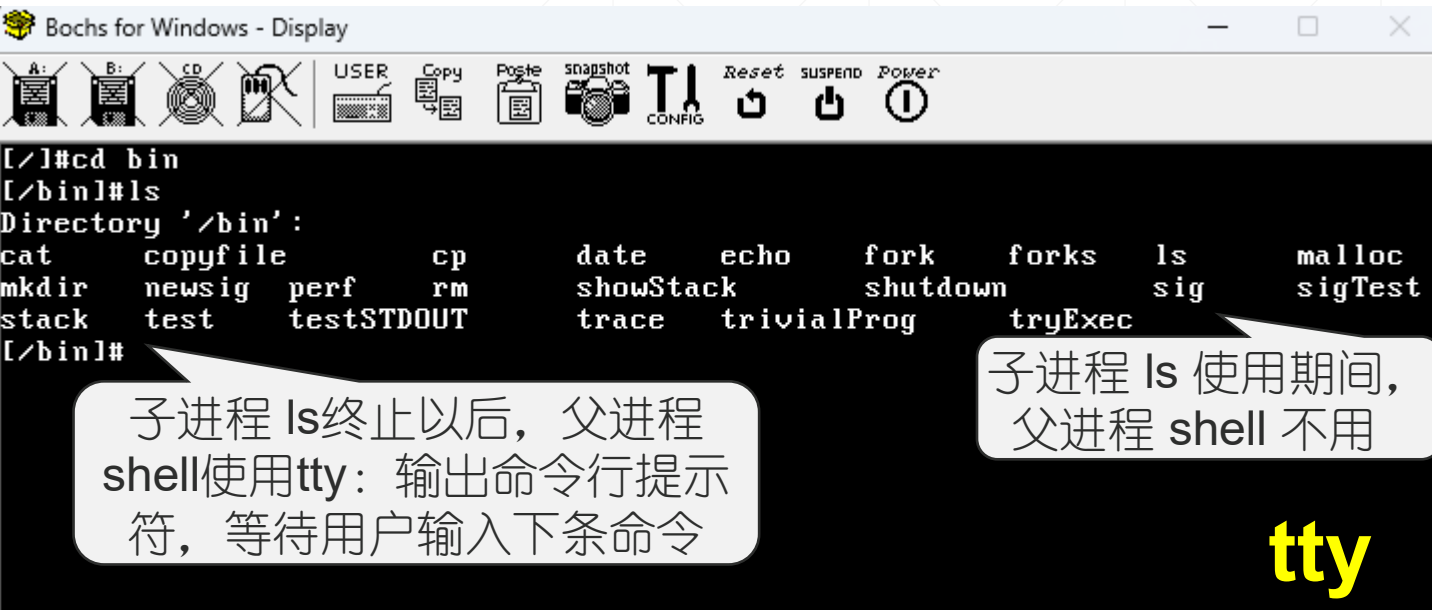
File[0]



File[1]



Inode[0] /dev/tty



Bochs for Windows - Display

USER Copy Paste snapshot CONFIG Reset SUSPEND Power

```

[/]#cd bin
[/bin]#ls
Directory '/bin':
cat      copyfile  cp        date      echo      fork      forks      ls        malloc    sigTest
mkdir    newsig     perf      rm        showStack shutdown  sig        sigTest
stack    test       testSTDOUT trace     trivialProg tryExec
[/bin]#
  
```

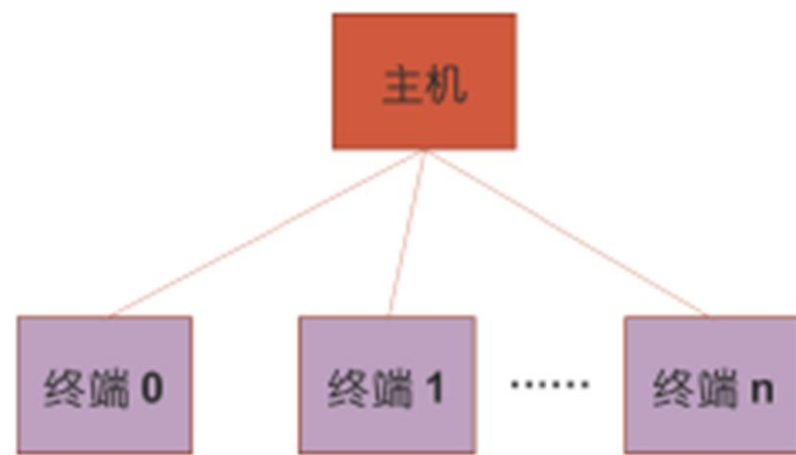
子进程 ls 终止以后，父进程 shell 使用 tty：输出命令行提示符，等待用户输入下条命令

子进程 ls 使用期间，父进程 shell 不用

tty

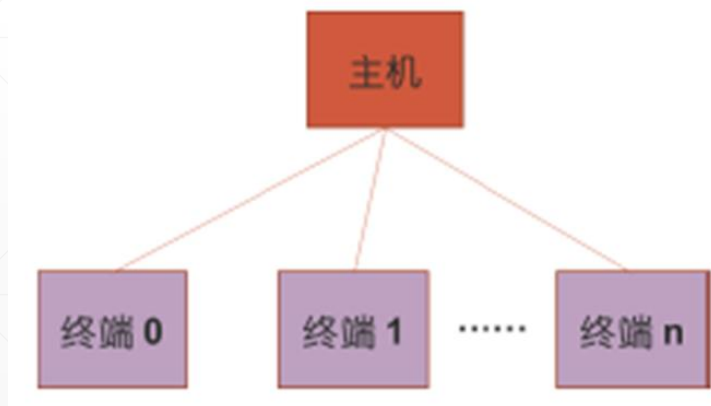
二、系统初始化

- (1) 将主硬盘上的引导扇区读入内存，执行其中代码，读入操作系统内核映像 (kernel.exe)
- (2) 跳转至内核入口main0函数。main0函数初始化系统
 - 读实时钟，得到当前时刻→time变量
 - 初始化硬件设备
 - 初始化所有内核模块
- (3) 创建0#进程
- (4) 0#进程fork创建1#进程
 - 1#进程执行应用程序init，是系统的监控程序。
 - 0#进程调用Sched()，入睡等待swap in任务（盘交换区上出现第一个就绪进程）。



1#进程 (监控程序 init)

- 1#进程为每个终端 fork 1个子进程 P_i 。完成后, 1#进程睡眠, 等待子进程终止。
- 子进程 P_i 管理终端 tty_i
 - (1) 初始化 tty_i : 为这个tty 分配 tty 结构 (含键盘输入缓存 和 屏幕输出缓存) ,
 - (2) 设置 $p_ttyp \rightarrow$ 这个缓存。
 - (3) 设置 标准输入输出文件
 - 0#文件: 标准输入文件, 引用键盘输入缓存
 - 1#文件: 标准输出文件, 引用屏幕输出缓存
 - 2#文件: 标准错误输出文件, 引用屏幕输出缓存
 - (4) `exec(login)` , 1#进程执行 login 程序。



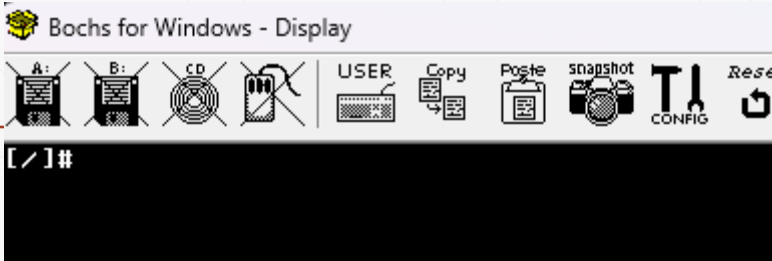
login程序

- 睡眠、等待用户输入用户名、口令字。
- 得到用户输入的用户名、口令字后，查花名册文件/etc/passwd，验证用户身份。

login: *****
Password: *****

user name (in ASCII) (string)	encrypted password	numerical user ID uid	numerical group ID gid	PATH 用户环境变量	initial working directory 工作目录	initial shell (UI program) 用户界面
root	X	0	0	*****	/root	/bin/sh
.....						
John	X	12	1	*****	/home/John	/bin/sh
Mary	X	13	1	*****	/home/Mary	/bin/sh
.....						
Tony	X	58	1	*****	/home/Tony	/bin/x_windows

- 有匹配记录，验证通过。进程 Pi
- 设置 p_uid = uid
 - 设置 u_dirp = 用户的初始工作目录
这是这次会话，相对路径名的搜索起点
 - 用户栈底登记环境变量。
环境变量名 = 值，字符串列表
 - 初始化终端。
 - execve(shell,argv,envp)成为shell进程，为用户提供命令行界面服务。
 - shell进程输出命令行提示符，随后入睡，等待用户输入。



shell程序 (简版框架)

读取用户输入的命令行, 启动应用程序

```
While( )
{
    printf("$");
    scanf("%s", cmdLine); //读取用户输入的命令行

    parse(cmdLine, argv); // 解析cmdLine, argv存解析出来的字符串

    i = fork( ); // 创建一个子进程
    if(i)
        wait( ); ..... // shell进程睡眠等待子进程终止
    else
        exec(argv[0], argv); // 子进程执行命令行指示的应用程序
}
```



The image shows a screenshot of the Bochs for Windows - Display window. The window title is "Bochs for Windows - Display". The interface includes a toolbar with icons for A:, B:, CD, USER, Copy, Paste, snapshot, CONFIG, Reset, SUSPEND, and Power. The main display area shows a shell prompt [/bin]# and the following commands and output:

```
[ /bin]#trace off
trace OFF
[ /bin]#trivialProg arg1 arg2 arg3 arg4 arg5 arg6 arg7
argument 0:    trivialProg
argument 1:    arg1
argument 2:    arg2
argument 3:    arg3
argument 4:    arg4
argument 5:    arg5
argument 6:    arg6
argument 7:    arg7
[ /bin]#_
```

执行应用程序

每执行一个应用程序，就会派发一个子进程

```
While( )
{
    printf("$");
    scanf("%s", cmdLine); //读取用户输入的命令行

    parse(cmdLine, argv); // 解析cmdLine, argv存解析出来的字符串

    i = fork( ); // 创建一个子进程
    if(i)
        wait( ); ..... // shell进程睡眠等待子进程终止
    else
        exec(argv[0], argv); // 子进程执行命令行指示的应用程序
}
```

fork, 复制shell进程。

- p_uid, u_dirp, p_ttyp与父进程相同
- 代码段、数据段、用户栈里是shell程序

子进程执行exec系统调用，刷新用户空间。

- p_uid, u_dirp, p_ttyp不变
- 代码段、数据段、用户栈里是目标应用程序



所以，应用程序执行时，操作系统知道执行它的用户，知道它使用的tty终端，知道它的当前工作目录（全是从shell进程那里来的）

- `p_uid`，用来确定应用程序资源访问权限。
- 当应用程序需要打开文件时，如果使用了相对路径名，`u_dirp`是目录搜索起点。
- `scanf`从`p_ttyp`的输入缓存读数据，`printf`将字符写入输出缓存供屏幕显示。

tty使用协议

子进程执行期间，**shell**睡眠，屏幕和键盘归子进程使用

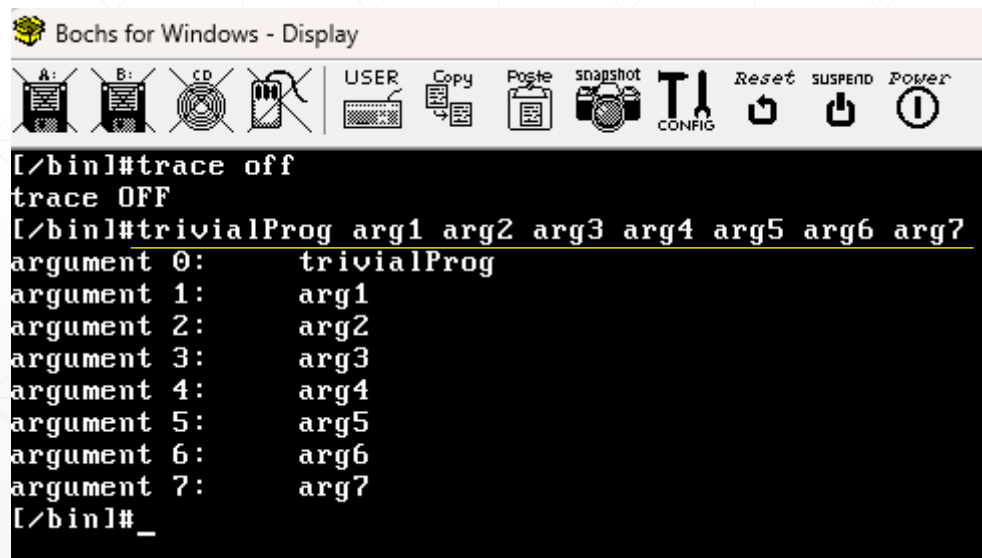
- 子进程读取键盘输入，向屏幕输出。

应用程序终止后，唤醒**shell**进程。屏幕和键盘归**shell**进程使用

- **shell**进程输出命令行提示符，等待用户输入下个命令行。

```
While( )
{
    printf("$");
    scanf("%s", cmdLine); //读取用户输入的命令行

    // 解析cmdLine, argv存解析出来的字符串
    parse(cmdLine, argv);
    i = fork( );
    if(i)
        wait( ); .....
    else
        exec(argv[0], argv);
}
```



The image shows a screenshot of the Bochs for Windows - Display window. The window title is "Bochs for Windows - Display". The toolbar includes icons for A:, B:, CD, USER, Copy, Paste, Snapshot, CONFIG, Reset, Suspend, and Power. The command prompt shows the following text:

```
[/bin]#trace off
trace OFF
[/bin]#trivialProg arg1 arg2 arg3 arg4 arg5 arg6 arg7
argument 0:      trivialProg
argument 1:      arg1
argument 2:      arg2
argument 3:      arg3
argument 4:      arg4
argument 5:      arg5
argument 6:      arg6
argument 7:      arg7
[/bin]#_
```



进程终止

- 目录
 - 进程终止的原因
 - 子进程执行的exit系统调用 和 父进程执行的wait系统调用
 - 一些例题
 - Unix的进程树

进程终止的原因——正常终止

- 应用程序运行完毕，进程main返回 或 执行exit系统调用。

```
// 例：echo.c 程序
#include <stdio.h>
#include <stdlib.h>
int main1(int argc, char* argv[])
{
    .....
    主函数或任何子程序中执行exit(n)系统调用
    .....
}
```

返回码：n

```
// 例：date.c 程序
#include <stdio.h>
#include <stdlib.h>
int main1(int argc, char* argv[])
{
    .....
    主函数任意位置return m;
    .....或 无return语句
}
```

返回码：0

```
extern "C" void runtime()
{
    __asm__ __volatile__("    leave;  \n\n    movl %%esp, %%ebp;  \n\n    call *%%eax;        \n\n    movl $1, %%eax; \n\n    movl $0, %%ebx; \n\n    int $0x80::");
}
```

进程终止的原因——异常终止

- 应用程序没必要继续运行
 - 1、用户使用 `ctrl+c` 终止前台运行的死循环程序
 - 2、用户执行命令 `kill -9 pid`, 杀死目标进程 `pid`
 - 3、发送信息时socket对端, 接收进程关闭链接
 - 4、程序bug, 比如非法内存访问, 除数为0
- Unix使用信号机制, 向进程发信号。收到信号进程, 异常终止。我们说, 信号杀死进程。
 - 1、SIGINT信号 (2#)
 - 2、SIGKILL信号 (9#)
 - 3、SIGPIPE信号 (13#)
 - 4.1 非法内存访问, SIGSEGV信号 (11#)
 - 4.2 除数为0, SIGFPE信号 (8#)

被信号杀死的进程,
终止码是杀死它的信号

终止进程使用的内核数据结构

```
class Process
{
    .....
    int p_sig; // 收到的信号
    .....
}
```

```
class User
{
    .....
    int u_arg[0]; // 终止码
    int u_utime; // 用户态运行时长
    int u_stime; // 核心态运行时长
    int u_cutime; // 子进程用户态运行时长总和
    int u_cstime; // 子进程核心态运行时长总和
    .....
} // 终止子进程的时长，需要累加进父进程
```

Unix 的进程终止操作——正常终止

■ 进程执行exit系统调用

```
// 例: echo.c 程序
#include <stdio.h>
#include <stdlib.h>
int main1(int argc, char* argv[])
{
    .....
    主函数或任何子程序中执行exit(n)系统调用
    .....
}
```

```
int exit(int status) /* 子进程返回给父进程的Return Code */
{
    int res;
    __asm__ __volatile__ ( "int $0x80":"=a"(res):"a"(1),"b"(status));
    if ( res >= 0 )
        return res;
    return -1;
}

int SystemCall::Sys_Rexit()
{
    User& u = Kernel::Instance().GetUser();
    u.u_arg[0] = u.u_arg[0] << 8;
    u.u_procp->Exit();
    return 0; /* GCC likes it ! */
}
```

终止码是返回码左移8位

Unix 的进程终止操作——异常终止

- 收到信号的进程终止自己

```
void Process::PSig(struct pt_context* pContext) // 进程处理收到的信号
{
    User& u = Kernel::Instance().GetUser();
    int signal = this->p_sig;

    if ( u.u_signal[signal] != 0 ) // 怎样处理信号 ?
        ..... 2、如果有定义，返回用户空间执行信号处理函数.....
    else { // 1、Unix默认的信号处理方式：终止自己
        u.u_arg[0] = signal; // 终止码是杀死进程的信号
        u.u_procp->Exit();
    }
}
```

子进程的Exit() 和 父进程的Wait()

- 子进程执行Exit()函数终止。
 - 释放所有资源，留PCB（Process结构和User结构）。
 - 唤醒父进程，回收PCB。
- 父进程执行wait系统调用，回收终止子进程的PCB。每执行一次，回收一个PCB。

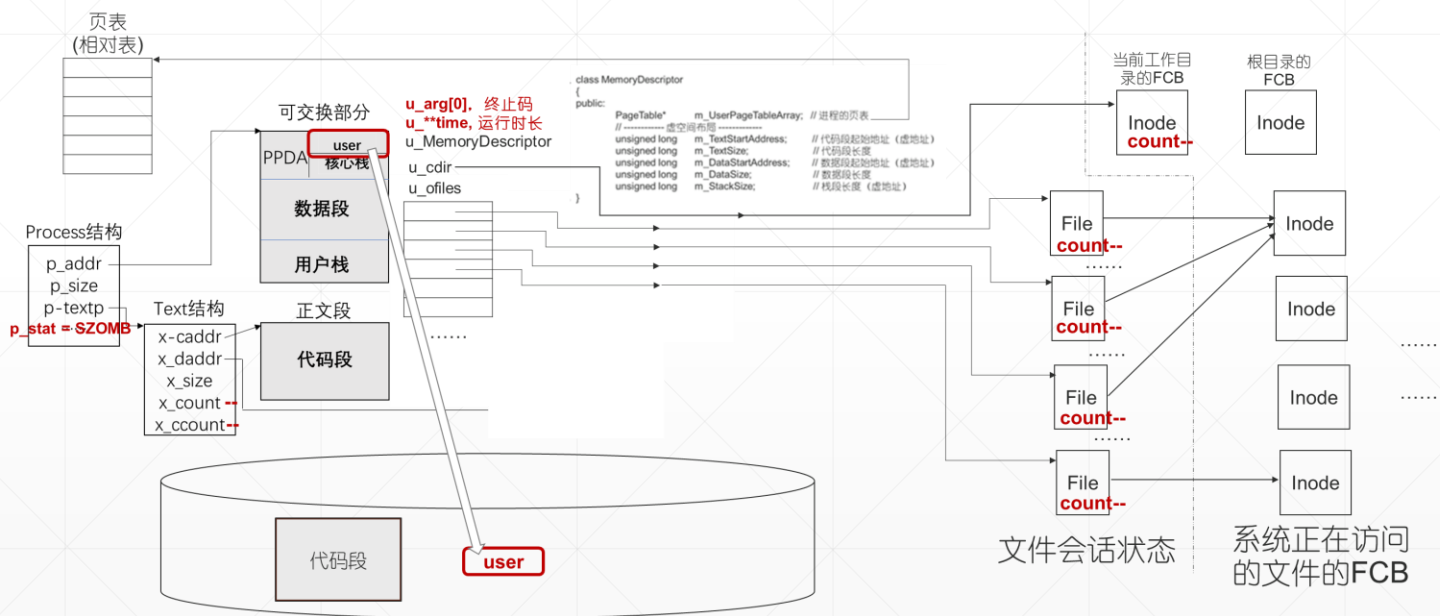
int pid;

int status;

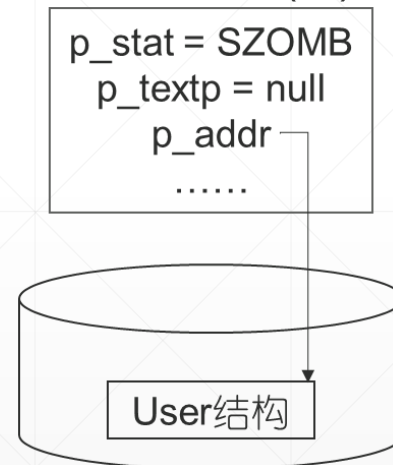
pid = wait(&status);

- 入口参数是一根指针，指向用户空间的变量status。
- 返回值是终止子进程的pid（PCB已回收）。
- 父进程执行wait系统调用的时候，如果没有子进程终止，睡眠等待

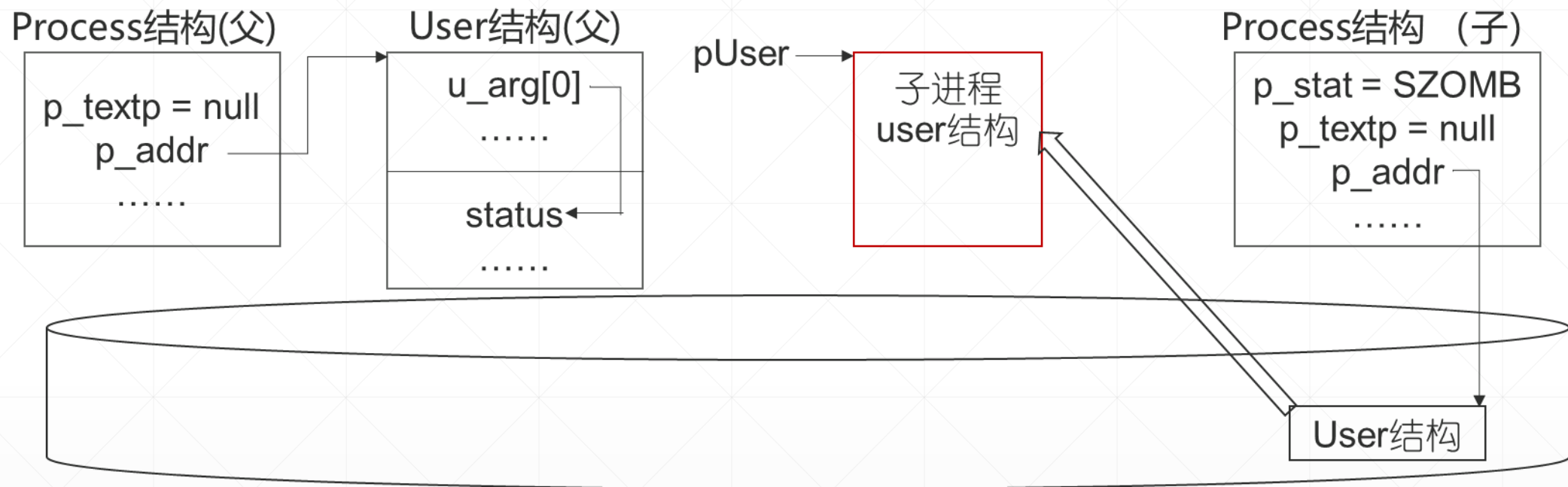
细节: Exit()



Process结构 (子)



细节: Wait()



- 1、`u_arg[0] = 子进程pid`
- 2、申请一块缓存，读入磁盘上子进程的user结构
- 3、累加子进程运行时长
- 4、释放子进程的Process结构，盘交换区上的User结构。释放缓存块。



源代码 Exit()

```
void Process::Exit()
{ .....
    /* 1、信号处理方式设置为1，不再响应任何信号 */
    for ( i = 0; i < User::NSIG; i++ )
    {
        u.u_signal[i] = 1;
    }

    /* 2、关闭所有的打开文件 */
    for ( i = 0; i < OpenFiles::NOFILES; i++ )
    {
        File* pFile = NULL;
        if ( (pFile = u.u_ofiles.GetF(i)) != NULL )
        {
            fileTable.CloseF(pFile);    //释放File结构 (引用计数--)
            u.u_ofiles.SetF(i, NULL);    //打开文件表相关元素 (fd) 置null
        }
    }
}
```



```
/* 访问不存在的fd会产生error code, 清除u.u_error避免影响后续程序执行流程 */
```

```
u.u_error = User::NOERROR;
```

```
/* 3、当前工作目录的引用计数-- */
```

```
inodeTable.IPut(u.u_cdir);
```

```
/* 4、释放代码段（引用计数减 1，减至 0 释放占用的内存空间和盘交换区空间） */
```

```
if ( u.u_procp->p_textp != NULL )
```

```
{
```

```
    u.u_procp->p_textp->XFree();
```

```
    u.u_procp->p_textp = NULL;
```

```
}
```

```
/* 5、user结构写入盘交换区 */
```

```
/* 1、盘交换区申请一个扇区（512字节），扇区号是blkno*/
```

```
int blkno = swapperMgr.AllocSwap(BufferManager::BUFFER_SIZE);
```

```
.....
```

```
/* 2.1 内存申请一个缓存块（512字节），用来同步磁盘扇区blkno。磁盘IO要用到。起始地址pBuf->b_addr */
```

```
Buf* pBuf = bufMgr.GetBlk(DeviceManager::ROOTDEV, blkno);
```

```
/* 2.2 把user结构写进这个缓存块 */
```

```
Utility::DWordCopy((int *)&u, (int *)pBuf->b_addr, BufferManager::BUFFER_SIZE / sizeof(int));
```

```
/* 3 把这个缓存块同步写入磁盘。进程睡眠等待IO完成
```

```
bufMgr.Bwrite(pBuf);
```




```
/* 6、释放相对虚实地址映射表 */  
u.u_MemoryDescriptor.Release();
```

```
/* 7、释放可交换部分 */  
Process* current = u.u_procp; // 现运行进程的Process结构  
UserPageManager& userPageMgr = Kernel::Instance().GetUserPageManager();  
userPageMgr.FreeMemory(current->p_size, current->p_addr);
```

```
/* 8、p_addr指向磁盘上的user结构。置终止状态 */  
current->p_addr = blkno;  
current->p_stat = Process::SZOMB;
```

```
/* 9、唤醒父进程 */
for ( i = 0; i < ProcessManager::NPROC; i++ )
{
    if ( procMgr.process[i].p_pid == current->p_ppid ) // 父进程PID
    {
        procMgr.WakeUpAll((unsigned long)&procMgr.process[i]);
        break;
    }
}

/* 10、没找到父进程（父进程已经终止） */
if ( ProcessManager::NPROC == i )
    current->p_ppid = 1;
```



/* 11、将子进程传给1#进程 */

```
for ( i = 0; i < ProcessManager::NPROC; i++ )
```

```
{
```

```
    if ( current->p_pid == procMgr.process[i].p_ppid )    // 找到子进程
```

```
    {
```

```
        Diagnose::Write(".....",.....); // 内核使用的格式化输出函数，相当于应用程序用的printf
```

```
        procMgr.process[i].p_ppid = 1;    // 把它们的父进程改成1#进程
```

```
        if ( procMgr.process[i].p_stat == Process::SSTOP )
```

```
        {
```

```
            procMgr.process[i].SetRun();
```

```
        }
```

```
    }
```

```
}
```

/* 12、唤醒1#进程，回收孤儿进程 */

```
procMgr.WakeUpAll((unsigned long)&procMgr.process[1]);
```

```
procMgr.Switch( );    // 13、放弃CPU
```

```
}
```



Unix V6 的 exit()

```
3219 exit()
3220 {
3221     register int *a, a;
3222     register struct Proc *p;
3223
3224     u.u_procp->p_flag = & ~STRC;
3225     for(a = &u.u_signal[0]; a < &u.u_signal[NSIG];)
3226         *a++ = 1;
3227     for(a = &u.u_ofile[0]; a < &u.u_ofile[NOFILE]; a++)
3228         if(a = *a) {
3229             *a = NULL;
3230             closef(a);
3231         }
3232     iput(u.u_cdir);
3233     xfree();
3234     a = malloc(swampmap, 1);
3235     if(a == NULL)
3236         panic("out of swap");
3237     p = getblk(swampdev, a);
3238     bcopy(&u, p->b_addr, 256);
3239     bwrite(p);
3240     a = u.u_procp;
3241     mfree(coremap, a->p_size, a->p_addr);
3242     a->p_addr = a;
3243     a->p_stat = SZOMB;
3244
3245 loop:
3246     for(p = &Proc[0]; p < &Proc[NPROC]; p++)
3247         if(a->p_ppid == p->p_pid) {
3248             wakeup(&Proc[1]);
3249             wakeup(p);
```

```
for(p = &Proc[0]; p < &Proc[NPROC]; p++)
if(a->p_pid == p->p_ppid) {
    p->p_ppid = 1;
    if (p->p_stat == SSTOP)
        setrun(p);
}
switch();
/* no return */
}
a->p_ppid = 1;
goto loop;
```



源代码 Wait()

```
void ProcessManager::Wait()
{
    while(true)
    {
        for ( i = 0; i < NPROC; i++ )
        {
            if ( u.u_procp->p_pid == process[i].p_ppid ) /* 找所有的子进程 */
            {
                hasChild = true;
                if( Process::SZOMB == process[i].p_stat ) /* 回收一个终止子进程的PCB */
                {
                    处理终止的子进程;    return;    } /* PPT 21*/
            }
        }
        if (true == hasChild) // 存在尚未终止的子进程
        {
            Diagnose::Write("wait until child process Exit! ");
            u.u_procp->Sleep((unsigned long)u.u_procp, ProcessManager::PWAIT); // 入睡，等待子进程终止
            continue; /* 唤醒后，再进for循环，找终止子进程 */
        }
        else
        {
            u.u_error = User::ECHILD;
            break; /* Get out of while loop */
        }
    }
}
```

处理终止的子进程 1

```
/* 准备wait()系统调用的返回值：子进程的pid */
```

```
u.u_ar0[User::EAX] = process[i].p_pid;
```

```
/* 释放 Process结构 */
```

```
process[i].p_stat = Process::SNULL;
```

```
process[i].p_pid = 0;
```

```
process[i].p_ppid = -1;
```

```
process[i].p_sig = 0;
```

```
process[i].p_flag = 0;
```

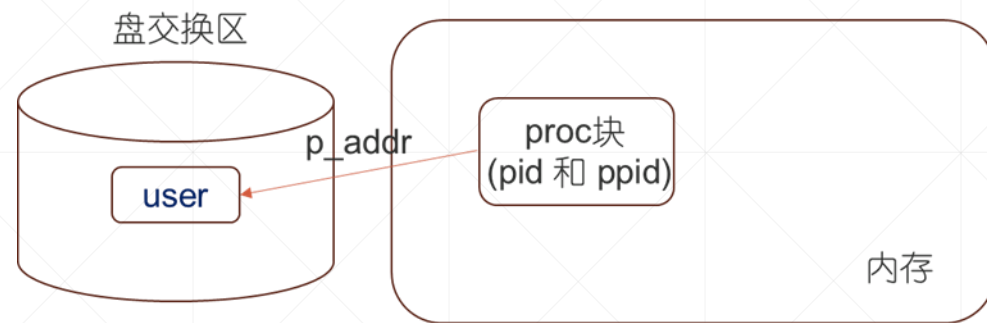
```
/* 启动IO，读入swap分区上的user结构，放在pBuf 管理的内存块 */
```

```
Buf* pBuf = bufMgr.Bread(DeviceManager::ROOTDEV, process[i].p_addr);
```

```
/* 释放盘交换区上user结构占用的空间 */
```

```
swapperMgr.FreeSwap(BufferManager::BUFFER_SIZE, process[i].p_addr);
```

```
User* pUser = (User *)pBuf->b_addr; // 内存块的首地址，就是读入的子进程user结构的首地址 pUser
```



处理终止的子进程 2

```
/* 终止子进程的时间累加到父进程上，在字段u_c*time里 */
```

```
u.u_cstime += pUser->u_cstime + pUser->u_stime;
```

```
u.u_cutime += pUser->u_cutime + pUser->u_utime;
```

```
/* 指针 pInt = j */
```

```
int* pInt = (int *)u.u_arg[0];
```

```
i = wait( &j );
```

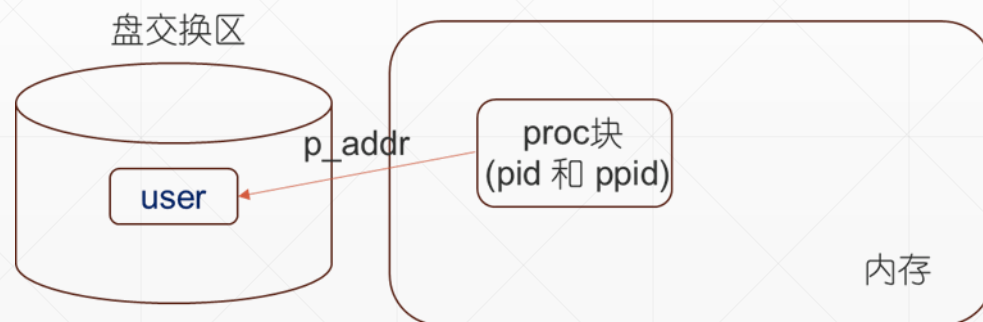
```
*pInt = pUser->u_arg[0]; // 子进程终止码写入用户空间，变量 j
```

```
/* 释放pBuf 管理的内存块 */
```

```
bufMgr.Brelse(pBuf);
```

```
return;
```

```
}
```



例题 1

已知， Unix V6++系统，只有这一个程序在执行。父进程pid==2，子进程pid==3。

问， (1) i = ?, j = ?

(2) 这个程序的输出是什么？

(3) 父进程会睡嘛？

```
#include <stdio.h>
#include <sys.h>
int main1(int argc, char* argv[])
{
    int i, j;
    if(fork())
    {
        i = wait(&j);
        printf("It is parent process. \n");           // 1
        printf("The finished child process is %d. \n", i); // 2
        printf("The exit status is %d. \n", j);         // 3
    }
    else
    {
        printf("It is child process. \n");             // 4
        sleep( 2 );
    }
}
```


例题 2

已知， Unix V6++系统， 只有这一个程序在执行。
父进程pid==2， 子进程pid==3。

问， (1) 这个程序的输出是什么？
(2) 父进程有可能不睡吗？

```
#include <stdio.h>
#include <sys.h>
int main1(int argc, char* argv[])
{
    int i, j;
    if(fork())
    {
        i = wait(&j);
        printf("It is parent process. \n");           // 1
        printf("The finished child process is %d. \n", i); // 2
        printf("The exit status is %d. \n", j);        // 3
    }
    else
    {
        printf("It is child process. \n");            // 4
    }
}
```



例题 3 用wait、exit系统调用实现任务之间的前驱后继关系

```
int main1(int argc, char* argv[])
{
    int i, j;
    if(fork())
    {
        i = wait(&j);
        后继任务。。。
    }
    else
    {
        前驱任务。。。
    }
}
```

例题 4

已知， Unix V6++系统，只有这一个程序在执行。父进程 `pid==2`，子进程1 `pid==3`，子进程2 `pid==4`。

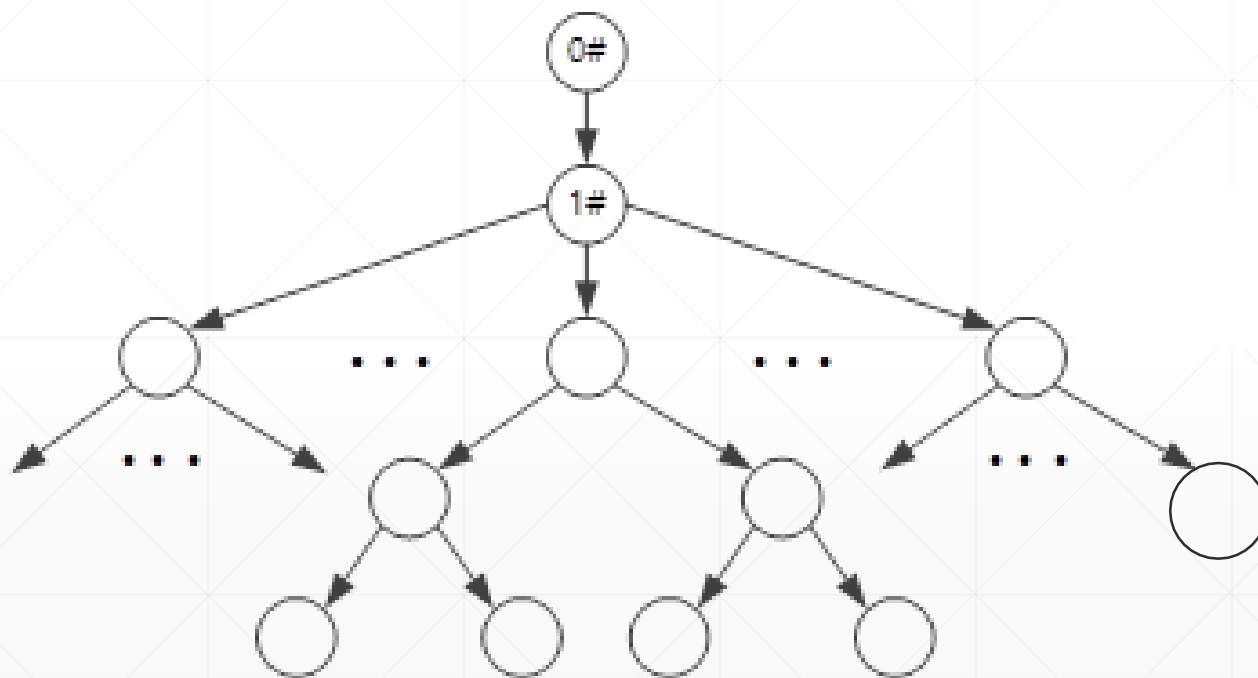
问， (1) 这个程序的输出是什么？
(2)子进程1 和 子进程2 的PCB分别是哪个进程回收的？
(3) 父进程终止后，PCB是哪个进程回收的？

```
#include <stdio.h>
#include <sys.h>

int main1(int argc, char* argv[])
{
    int i, j;
    if(fork())
    {
        printf("father. \n");
        if(fork())
        { i = wait(&j); printf(" exit child = %d, exit status = %d.\n", i, j)>>8) }
        else
        { printf("second child. \n"); exit(3); }
    }
    else
    {
        sleep( 2 );
        printf("first child. \n");
        exit(5);
    }
}
```

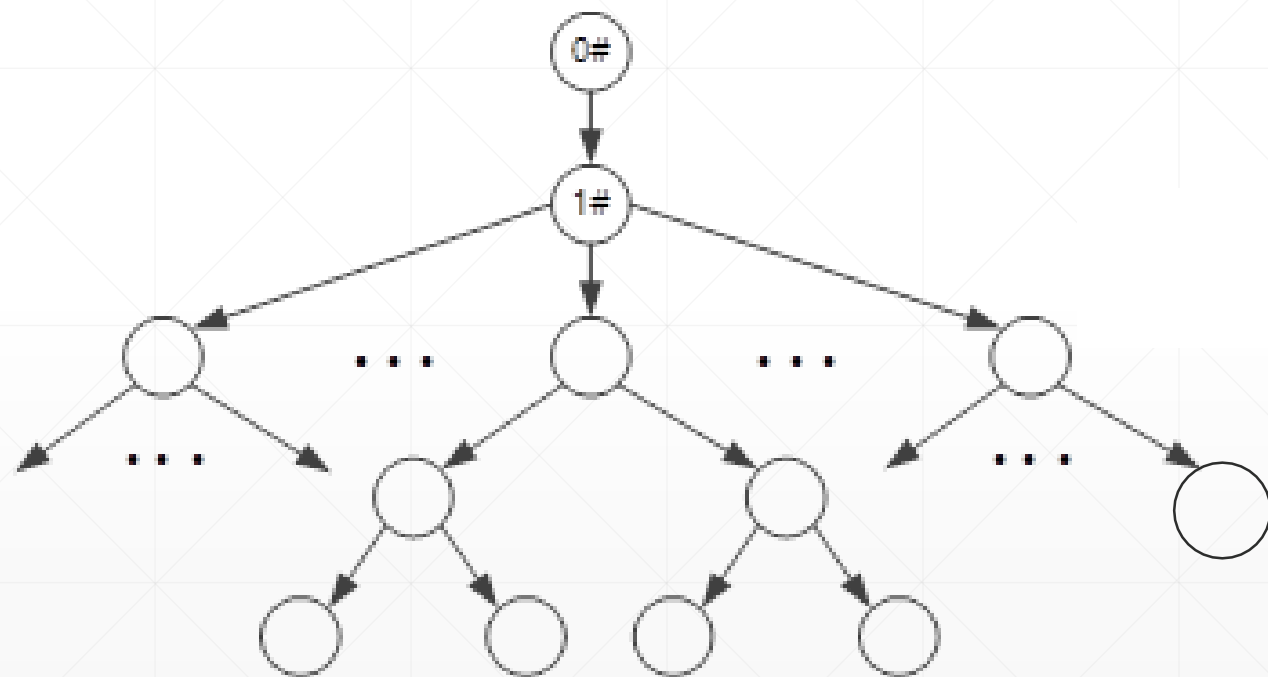
Part 2 Unix 进程树 和 shell

- 在Unix系统中，除0#进程外，**所有进程都是fork创建出来的**。
- 执行fork的是父进程，被创建的是子进程。进程之间的父子关系，绘成进程树。



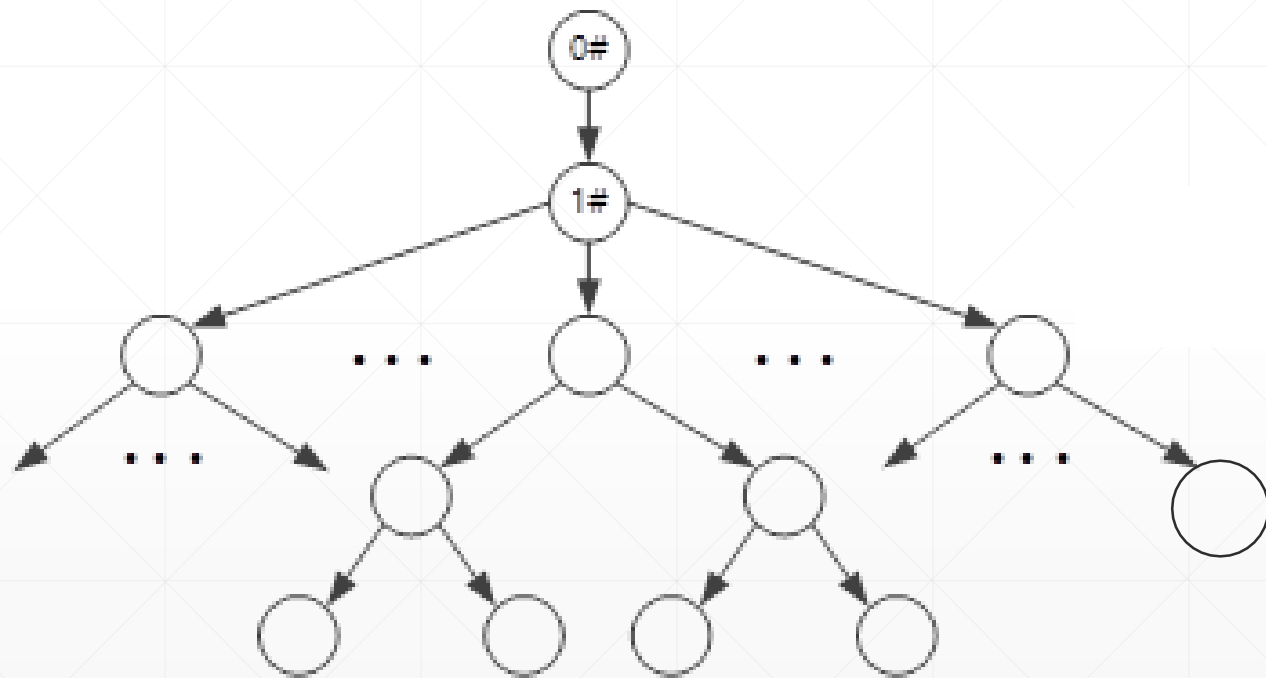
不同的进程执行不同任务

- **0#进程**, 又叫 **idle进程** (1) 调度, 选新运行进程 (2) CPU空转时, 响应中断, 执行中断处理程序。0#进程是内核线程, 负责系统维护, 保障系统正常运行。永不终止。
- **1#进程**, 又叫 **init进程** (1) 监控终端, 为每个加电终端创建1个shell进程 (2) 回收孤儿进程的PCB。1#进程是精灵进程 (daemon), 这个用户态服务器进程, 永不终止。
- **shell进程**, 为用户提供命令行界面, 驱动应用程序。用户logout, 进程终止。
- **其余, 普通进程**。每个进程执行一个应用程序。应用程序执行完毕, 进程终止。



进程终止

- **每个进程承担一项任务。任务完成，进程终止。**
- 0#进程和其它内核线程，是内核服务器，永不终止。
- 1#进程是精灵进程。永不终止。
- shell进程。会话结束（用户logout），shell进程终止。
- 其余进程，应用程序执行完毕，进程终止。
- **进程终止后，系统唤醒父进程或1#进程，执行wait系统调用，回收PCB。**





Unix系统中的作业 (job)

- 用户通过命令行提交作业。
- 前台作业需要与用户交互，优先级高

`$ copy file1 file2`

- 后台作业无需与用户交互，优先级低

`$ copy file1 file2 &`

- 作业，可以包含多个应用程序。
 - 例：包含2个程序的作业 `$ program1 | program2`
Unix创建2个进程，program1进程 和 program2进程，并且建立者两个进程之间的管道：program1进程的标准输出 是 program2进程的标准输入。
- 作业控制：
 - 杀死前台作业的进程：用户ctrl+c，用SIGINT信号杀死前台作业的进程。
 - 杀死后台作业的进程：kill -9 pid，用SIGKILL信号杀死后台作业的进程。



- shell进程睡眠等待前台作业终止（最先创建的那个进程），获得终止码。

```
main( )
```

```
{
```

```
.....
```

```
while( )
```

```
{
```

输出 \$, 睡眠等待用户输入命令行: command arg1 arg2

如果输入的是 "logout", shell进程就exit

```
while((i=fork( ))== -1);
```

```
if( ! i)
```

```
    exec( "command" ,arg1, arg2, .....);
```

```
else if ( 命令行中没有后台命令符号& ) {
```

```
    child = wait(&terminationStatus);
```

```
    Switch (terminationStatus & 0xFF)
```

```
        case SIGSEGV: printf(" Segment Fault" ); break;
```

```
        .....
```

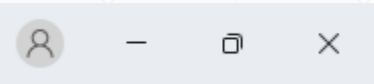
```
    } //shell进程不会睡眠等待后台进程终止
```

```
}
```

```
}
```

shell进程终止后，后台进程会继续运行。它们的父进程是1# 进程。后台进程终止后，1#进程回收其PCB 和 它的子进程的 PCB。

Part 3、Unix V6++的信号处理机制

- 信号是发送给目标进程的小整数。
- 信号的作用
 - 系统杀死buggy进程
 - 用户提前终止应用程序
 - CLI程序。ctrl+c终止前台进程
 - GUI程序。
 - 并行系统中，用信号实现应用系统层面控制流。
 - sender进程发送信号，receiver进程执行信号处理程序响应信号。



Unix V6++信号处理机制

```
class Process
```

```
{
```

```
.....
```

```
int p_sig;      // 进程收到的信号
```

```
TTY* p_ttyp;    // SIGINT要用到
```

```
// 进程收到的信号位图 (未用)
```

```
unsigned long p_sigmap;
```

```
.....
```

```
}
```

```
class User
```

```
{
```

```
.....
```

```
// 信号处理方式
```

```
unsigned long u_signal[NSIG];
```

```
.....
```

```
}
```

```
public:
```

```
// 32个POSIX传统信号, 定义于User.h
```

```
static const int NSIG = 32; /* 信号个数 */
```

```
static const int SIGNUL = 0;      /* No Signal Received */
```

```
static const int SIGHUP = 1;      /* Hangup (kill controlling terminal) */
```

```
static const int SIGINT = 2;      /* Interrupt from keyboard */
```

```
static const int SIGQUIT = 3;     /* Quit from keyboard */
```

```
static const int SIGILL = 4; /* Illegal instruction */
```

```
static const int SIGTRAP = 5;     /* Trace trap */
```

```
static const int SIGABRT = 6;     /* use abort() API */
```

```
static const int SIGBUS = 7;      /* Bus error */
```

```
static const int SIGFPE = 8;      /* Floating point exception */
```

```
static const int SIGKILL = 9;     /* Kill (can't be caught or ignored) */
```

```
static const int SIGUSR1 = 10;    /* User defined signal 1 */
```

```
static const int SIGSEGV = 11;    /* Invalid memory segment access */
```

```
static const int SIGUSR2 = 12;    /* User defined signal 2 */
```

```
static const int SIGPIPE = 13;    /* Write on a pipe with no reader, Broken pipe */
```

```
static const int SIGALRM = 14;    /* Alarm clock */
```

```
static const int SIGTERM = 15;    /* Termination */
```

```
static const int SIGSTKFLT = 16; /* Stack fault */
```

```
static const int SIGCHLD = 17; /* Child process has stopped or exited, changed */
```

```
static const int SIGCONT = 18; /* Continue executing, if stopped */
```

```
static const int SIGSTOP = 19; /* Stop executing */
```

```
static const int SIGTSTP = 20; /* Terminal stop signal */
```

```
static const int SIGTTIN = 21; /* Background process trying to read, from TTY */
```

```
static const int SIGTTOU = 22; /* Background process trying to write, to TTY */
```

```
static const int SIGURG = 23; /* Urgent condition on socket */
```

```
static const int SIGXCPU = 24; /* CPU limit exceeded */
```

```
static const int SIGXFSZ = 25; /* File size limit exceeded */
```

```
static const int SIGVTALRM = 26; /* Virtual alarm clock */
```

```
static const int SIGPROF = 27; /* Profiling alarm clock */
```

```
static const int SIGWINCH = 28; /* Window size change */
```

```
static const int SIGIO = 29; /* I/O now possible */
```

```
static const int SIGPWR = 30; /* Power failure restart */
```

```
static const int SIGSYS = 31; /* invalid sys call */
```



信号处理方式

- **u_signal[i]**
 - 0, i#信号杀死进程，这是默认的信号处理方式。
 - 奇数，忽略，即不处理 i#信号
 - 偶数（信号处理程序的首地址）。返回用户态，执行信号处理程序。
- 应用程序刚开始执行时，u_signal[i]清0 ∴ 任何信号会杀死信号接收者进程。
- 应用程序执行**signal系统调用**，设置信号处理方式。
 - signal(信号 i, 信号 i 的处理函数); u_signal[i] = 信号 i 的处理函数
 - 收到 i#信号 后，进程会尽快返回用户态执行信号处理程序。信号处理程序执行完毕、返回后，进程继续执行被信号打断的任务。

用ctrl+c杀死进程

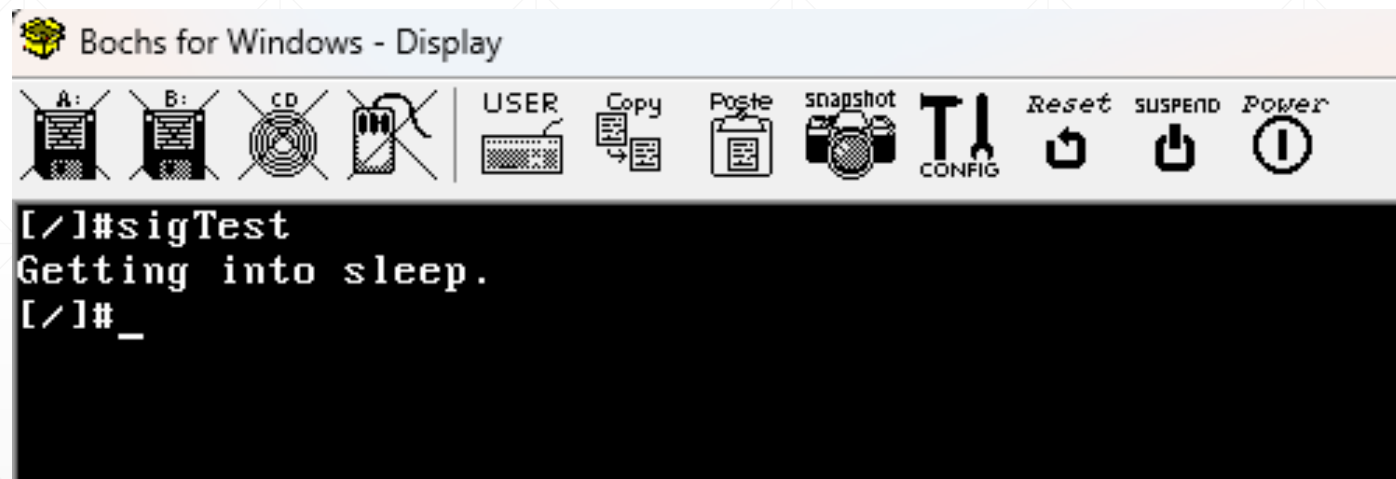
```
#include <stdio.h>
#include <sys.h>
```

```
/*test for ctrl-c*/
int main1(int argc, char* argv[])
{
    printf("Getting into sleep.\n");

    sleep(50);

    printf("Wakeup.\n");

    return 1;
}
```



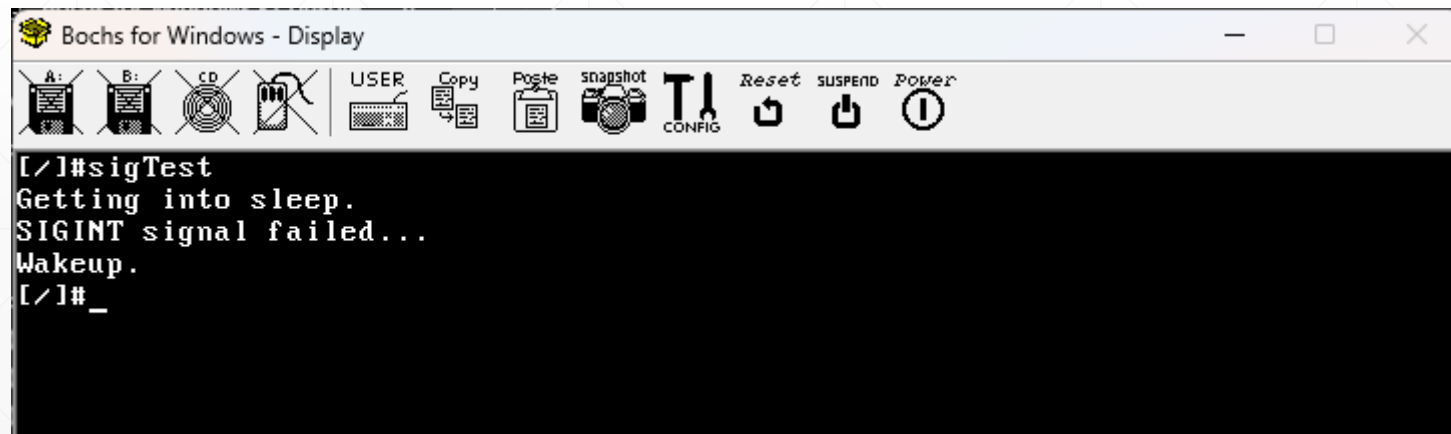
设置SIGINT的信号处理函数

```
#include <stdio.h>
#include <sys.h>

void SIGINT_Handler()
{
    printf("SIGINT signal failed...\n");
}

int main1(int argc, char* argv[])
{
    signal(SIGINT, SIGINT_Handler);

    printf("Getting into sleep.\n");
    sleep(50);
    printf("Wakeup.\n");
    return 1;
}
```

A screenshot of a Bochs for Windows - Display window. The window title is "Bochs for Windows - Display". The toolbar includes icons for A:, B:, CD, USER, Copy, Paste, snapshot, CONFIG, Reset, SUSPEND, and Power. The main display area shows the following text:

```
[/]#sigTest
Getting into sleep.
SIGINT signal failed...
Wakeup.
[/]#_
```

发信号

- kill -9 26
- 应用程序执行kill系统调用，向26#进程发SIGKILL信号（9#信号）

```
int SystemCall::Sys_Kill()
{
    ProcessManager& procMgr = Kernel::Instance().GetProcessManager();
    procMgr.Kill();
    return 0; /* GCC likes it ! */
}

void ProcessManager::Kill()
{ .....
    int pid = u.u_arg[0]; // 26
    int signal = u.u_arg[1]; // 9
    bool flag = false;

    for ( int i = 0; i < ProcessManager::NPROC; i++ )
    { .....
        if ( pid != 0 && process[i].p_pid == pid)
            process[i].PSignal(signal);
        .....
    }
}
```



```
void Process::PSignal( int signal )
{
    .....
    if ( this->p_sig != User::SIGKILL )
    {
        this->p_sig = signal; // 1、新的信号可以覆盖位被处理的旧信号，但SIGKILL不容覆盖，一定会杀死接受进程
    }

    if ( this->p_pri > ProcessManager::PUSER )
    {
        // 2、提高信号接收进程的优先权，至应用程序最高优先权。除非它正在执行系统调用下半段
        this->p_pri = ProcessManager::PUSER;
    }

    if ( this->p_stat == Process::SWAIT )
    {
        this->SetRun(); // 3、若进程低优先权睡眠，将其唤醒
    }
}
```


被信号打断的慢系统调用

```
void SystemCall::Trap1(int (*func)())
{
    User& u = Kernel::Instance().GetUser();

    u.u_intflg = 1;
    SaveU(u.u_qsav);
    func();
    u.u_intflg = 0;
}

void Process::Sleep(unsigned long chan, int pri)
{
    User& u = Kernel::Instance().GetUser();
    ProcessManager& procMgr = Kernel::Instance().GetProcessManager();

    if ( pri > 0 )
    {
        ..... 入睡操作, Swtch(); // 低优先级进程被唤醒后, Swtch()返回
        if ( this->IsSig() ) // 睡眠被信号打断
        {
            aRetU(u.u_qsav);
            return;
        }
    }
    // 正常情况: IO完成, 得到新数据; Sleep返回, 慢系统调用成功
    .....
}
```

信号接收进程，信号处理

- 信号接收进程故障。 异常处理程序同步检测信号并予以处理（下一节）。
- 其它原因收到信号，异步处理信号。信号处理时机：
 - 系统调用返回用户态。回看Trap()函数。
 - 整数秒，时钟中断返回用户态。

```
if ( current->IsSig() )  
{  
    current->PSig(context);  
}
```



```
int Process::IsSig()
{
    User& u = Kernel::Instance().GetUser();

    if ( this->p_sig == 0 )
    {
        return 0;    // 未收到信号
    }

    else if ( (u.u_signal[this->p_sig] & 1) == 0 )
    {
        return this->p_sig;
    }

    return 0;    // 未收到信号
}
```

```
void Process::PSig(struct pt_context* pContext)
{
    User& u = Kernel::Instance().GetUser();

    int signal = this->p_sig;
    this->p_sig = 0;

    if ( u.u_signal[signal] != 0 )
    { // 修改核心栈、用户栈。IRET返回用户态后，先
      执行信号处理程序，再执行被打断的应用程序。
        u.u_error = User::NOERROR;
        unsigned int old_eip = pContext->eip;
        pContext->eip = u.u_signal[signal];
        pContext->esp -= 4;
        int* plnt = (int *)pContext->esp;
        *plnt = old_eip;
        u.u_signal[signal] = 0;
        return;
    }
    u.u_procp->Exit();    // 信号杀死进程
}
```